

CHASKI: NAVEGACIÓN AUTÓNOMA EN EL LABERINTO

Henry Tovar Landa	Julia Rojas Estrada	Jordy Diaz Huanca	Daniel León Condori	Alex Perales Maldonado
Universidad ESAN	Universidad ESAN	Universidad ESAN	Universidad ESAN	Universidad ESAN
Lima, Perú	Lima, Perú	Lima, Perú	Lima, Perú	Lima, Perú
24100514@ue.edu.pe	24100478@ue.edu.pe	23101308@ue.edu.pe	17100703@ue.edu.pe	22200107@ue.edu.pe

Resumen—El Gran Prix CappyTown “El Laberinto del Chaski” plantea el desarrollo de un sistema de navegación autónoma capaz de recorrer un laberinto, respetar señales de tránsito y evitar obstáculos mediante percepción y control en tiempo real. Este trabajo presenta el diseño e implementación de la solución desarrollada por el Grupo 4 sobre una plataforma Yahboom Micro-ROS equipada con Raspberry Pi 5, LiDAR MS200 y cámara RGB, utilizando ROS 2 como entorno de integración.

La arquitectura propuesta se compone de tres nodos principales: `maze_solver`, encargado de la navegación mediante una máquina de estados finitos y un controlador PD para el seguimiento de paredes; `pare_detector`, responsable de la detección de señales PARE mediante segmentación en el espacio HSV y validación geométrica; y `box_detector`, utilizado para la detección y evasión de obstáculos.

El sistema integra la información proveniente del LiDAR y la cámara mediante una estrategia de arbitraje que prioriza la percepción visual para el reconocimiento de señales y el LiDAR para la navegación. Los resultados experimentales muestran un comportamiento estable en la navegación por pasillos, una detección confiable de señales PARE y el correcto funcionamiento de los módulos desarrollados sobre la pista física. Finalmente, se presentan las principales decisiones de diseño, las medidas de seguridad implementadas y el trabajo pendiente para la validación durante las pruebas oficiales de competencia.

Index Terms—navegación autónoma, robótica móvil, fusión sensorial, LiDAR, seguimiento de pared, máquina de estados finitos, segmentación HSV, ROS 2, CappyTown

I. INTRODUCCIÓN

El Qhapaq Ñan era el gran camino inca, una red de senderos que se bifurcaban entre montañas, con atajos, desvíos y caminos que no llevaban a ningún lado. El Gran Prix CappyTown retoma esa metáfora para el reto final del curso: un robot, al que llamamos “Karpinchu Chaski”, debe cruzar un laberinto de tablas de MDF desde el tambo de partida hasta la huaca de llegada, eligiendo su propia ruta sin que nadie lo maneje. El camino, eso sí, tiene reglas: en algunas intersecciones hay una señal de PARE, y un buen chaski se detiene por completo antes de seguir. Ganar no es solo llegar rápido, también se exige encontrar el camino más corto, respetar cada PARE y no derribar ninguna pared.

En el fondo, el reto es un problema de fusión sensorial con arbitraje, es decir, combinar la información de dos sensores

distintos y decidir cuál manda en cada momento. El LiDAR resuelve la navegación geométrica: ¿por dónde hay pasillo?, ¿dónde hay una esquina?, ¿estoy metido en un callejón sin salida? La cámara, en cambio, responde una pregunta que el LiDAR no puede contestar: ¿hay un cartel PARE en esta intersección? Y esa respuesta tiene que poder interrumpir la navegación geométrica aunque el pasillo esté físicamente despejado. El objetivo de este artículo es documentar, con el nivel de detalle técnico que se puede verificar directamente en el código fuente, la arquitectura, los algoritmos y las decisiones de diseño del sistema que implementó el Grupo 4 para este reto, junto con la evidencia experimental que recogimos durante las pruebas de validación sobre la pista física.

El resto del artículo se organiza así: la Sección II describe los Retos Clasificatorios previos cuyos nodos reutilizamos de forma explícita (uno de los requisitos del reto es “fusionar, no reprogramar percepción”). La Sección III detalla la arquitectura propuesta: los nodos de ROS 2, la máquina de estados de navegación, la ley de control de seguimiento de pared, el algoritmo de detección de PARE y la regla de arbitraje entre cámara y LiDAR. La Sección IV describe la configuración experimental (plataforma, pista, parámetros). La Sección V reporta los resultados de validación funcional obtenidos hasta el momento. Las Secciones VI y VII discuten las decisiones de diseño y cierran con las conclusiones y el trabajo pendiente antes de las corridas oficiales.

II. ANTECEDENTES: RETOS CLASIFICATORIOS REUTILIZADOS

El reto pide explícitamente no escribir percepción nueva salvo la estrictamente necesaria, sino fusionar piezas que ya construimos y depuramos en retos anteriores del curso. Heredamos tres piezas de forma directa.

II-A. Seguimiento de pared y censo de cajas (RC-4)

El controlador de seguimiento de pared, la detección de intersección o apertura y la recuperación en callejón sin salida del nodo `maze_solver` son una adaptación de `maze_navigator.py`, que desarrollamos para el Reto Clasificadorio “El Censo y el Guardián de las Cajas” (RC-4). De ese mismo reto reutilizamos, sin modificar su algoritmo, el

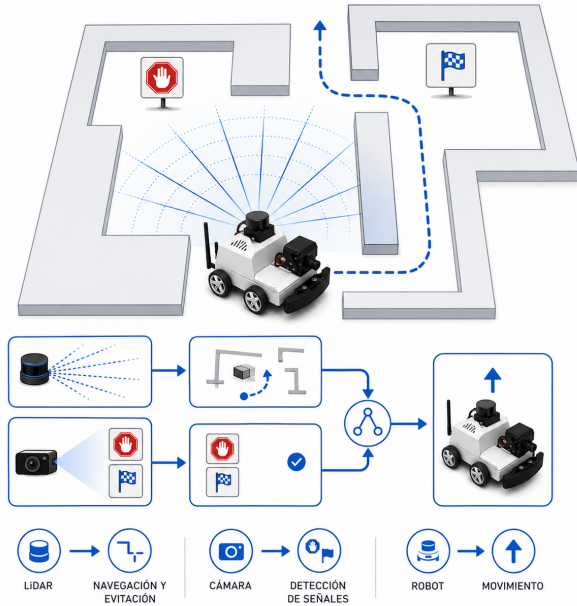


Figura 1. Metodología general: el LiDAR resuelve la navegación y evitación geométrica, la cámara resuelve la detección de señales (PARE/META), y ambas señales se arbitran antes de comandar el movimiento del robot.

nodo `box_detector`, que segmenta el `/scan` (la lectura cruda del LiDAR) en tramos continuos, identifica protuberancias de unos 20 cm de ancho como cajas (“karpinchus”) y las censa evitando contarlas dos veces gracias a su posición en el mundo, calculada con la odometría, que es la estimación de dónde está el robot a partir de cuánto se han movido sus ruedas.

II-B. Detección de señales por color (RC de visión)

La pieza de percepción por cámara (`pare_detector`) es nueva para este reto, ya que los Retos Clasificatorios anteriores del equipo usaban solamente el LiDAR. Sin embargo, reutiliza el mismo flujo de calibración HSV interactiva (sliders en vivo y la tecla `s` para guardar la configuración en un archivo YAML) que empleamos en el detector de carril/color del RC de visión, adaptado aquí a la geometría de un cartel en forma de octágono rojo en lugar de una franja de carril.

II-C. Extracción de líneas por Split & Merge

El módulo `split_merge.py` implementa el algoritmo clásico Split & Merge sobre las lecturas del LiDAR para extraer segmentos de recta, es decir, las paredes. En el Gran Prix lo incorporamos de forma aditiva dentro de `maze_solver` únicamente para registrar en el log cuántas líneas y cajas detecta cada ~ 2 s; no participa en ninguna decisión de la máquina de estados de navegación, que opera sobre sectores angulares simples del `/scan`. Lo conservamos por si se puede reutilizar en otro reto del curso centrado específicamente en Split & Merge.

III. ARQUITECTURA DEL SISTEMA PROPUESTO

La Fig. 1 resume esta metodología a alto nivel antes de entrar en el detalle por nodo.

III-A. Visión general de nodos

El sistema se implementa como el paquete de ROS 2 [1] `capytown_granprix_pkg` (`ament_python`). ROS 2 es el framework de robótica que usamos para que los distintos programas del robot, llamados nodos, se comuniquen entre sí mediante tópicos, que son básicamente canales de mensajes: un nodo publica en un tópico y otro se suscribe a él para recibir esos mensajes. Lanzamos cinco nodos ejecutables con `launch/granprix.launch.py`: `maze_solver`, `pare_detector`, `box_detector` (opcional, para la variante con karpinchus), `scan_map_viewer` (un visor en Tk, opcional) y `web_dashboard` (un panel HTTP opcional en el puerto 8080). La Fig. 2 resume el flujo de tópicos entre nodos.

III-B. Máquina de estados de navegación (`maze_solver`)

La navegación geométrica se implementa como una máquina de estados finitos (FSM): un esquema en el que el robot solo puede estar en un estado a la vez, por ejemplo “sigo la pared” o “estoy girando”, y salta de un estado a otro según lo que detecta. La implementamos como una función pura y testeable (`decide_state`, independiente de ROS), con las siguientes transiciones, evaluadas en este orden de prioridad sobre las distancias mínimas de los sectores `front`, `left` y `right`:

1. **Acorralado** (`front`, pared seguida y pared opuesta todas por debajo de `wall_block`) → `RECOVER` (giro de 180°).
2. **Apertura en la pared seguida** (`wall` $\geq$ `wall_lost`) → `TURN_OUT` (gira hacia la pared para tomar la esquina, regla de la mano derecha/izquierda según `side`).
3. **Frente bloqueado** (con histéresis y persistencia mínima `front_block_persist`) → `TURN_IN` (gira alejándose de la pared).
4. En cualquier otro caso → `FOLLOW_WALL`.

A estos cuatro estados geométricos se suma `PARAR_PARE` (forzado por la fusión con la cámara, que se explica en la Sección III-G) y los estados terminales o de seguridad `META`, `FAULT` y `DONE`. Los giros de $90^\circ/180^\circ$ se ejecutan en lazo cerrado por odometría (`run_turn`), no por tiempo: el robot gira hasta que el error angular respecto al `yaw` objetivo (la orientación del robot en el plano) cae bajo 4° , o hasta que el signo de ese error indica que ya se pasó del objetivo, lo que funciona como guarda contra el sobregiro, con un `timeout` de seguridad de `turn_timeout=6,0` s. Un detector de esquina alternativo (`corner_pose_aligned`) puede cortar el giro antes si la geometría del `/scan` (frente despejado, pared trasera visible, pared lateral seguida visible) ya corresponde a una pose de esquina correcta.

Un `watchdog` anti-cascada (`turn_in_max_accum_deg=135^\circ`) evita que una secuencia de giros `TURN_IN` de 90° sin avance real termine

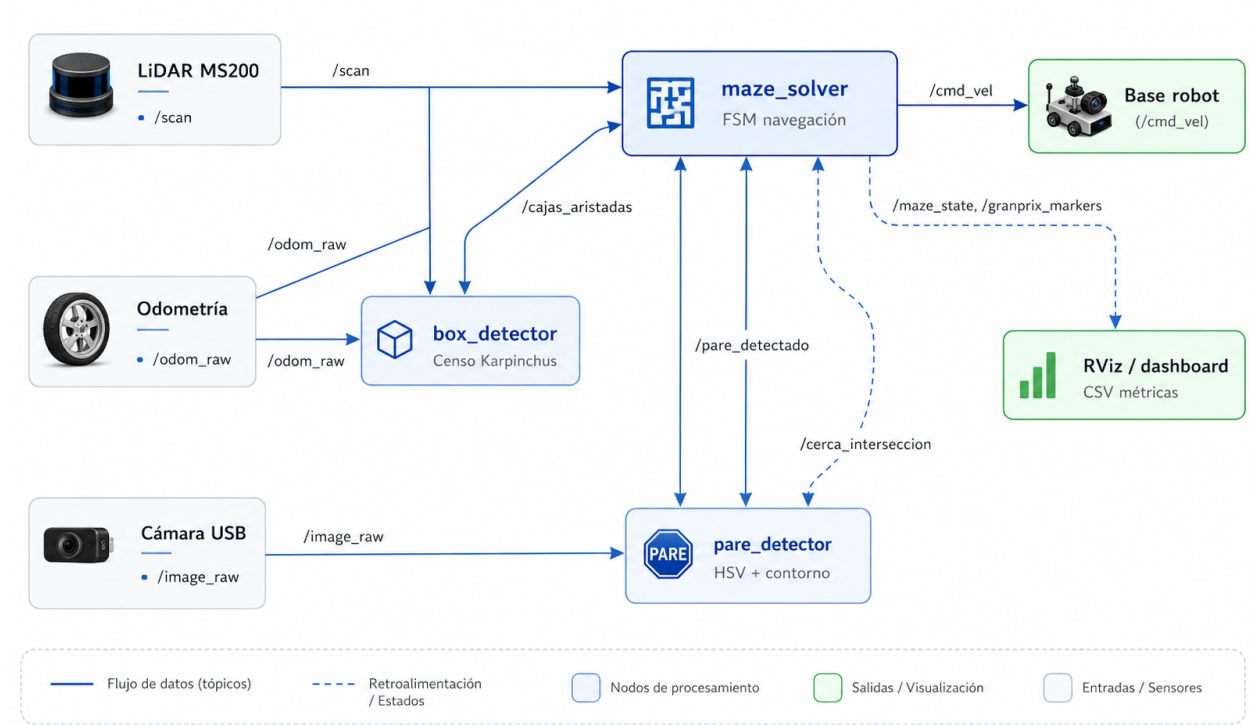


Figura 2. Arquitectura de nodos y tópicos ROS 2 del CapyTown Gran Prix. El LiDAR alimenta la geometría (`maze_solver`, `box_detector`); la cámara alimenta la semántica (`pare_detector`); `maze_solver` arbitra ambas señales (flujo discontinuo = retroalimentación/estado) y publica `/cmd_vel`.



Figura 3. Plataforma robótica utilizada: base Yahboom Micro-ROS con Raspberry Pi 5, LiDAR MS200 (torreta superior) y cámara sobre soporte pan orientable en el frente.

sumando 180° efectivos, un bucle que detectamos en las pruebas de campo. Si el acumulado supera ese umbral, el robot retrocede en vez de volver a girar, y el acumulador solo se reinicia después de `turn_in_reset_clear_t = 3,0s` con el frente sostenidamente despejado.

III-C. Ley de control de seguimiento de pared

El seguimiento de pared usa el esquema clásico de control proporcional-derivativo (PD): una ley de control que corrige el

error actual (término proporcional) y también su tendencia a crecer o achicarse (término derivativo), aplicada aquí sobre el error lateral respecto a la pared seguida. Es el esquema típico en robots móviles con sensores de rango [2], [3]:

$$e = d_{\text{obj}} - d_{\text{pared}}, \quad e_p = \begin{cases} 0 & |e| < \delta \\ e & \text{en otro caso} \end{cases} \quad (1)$$

$$\omega = \text{sgn}(k_p e_p + k_d \dot{e}), \quad \dot{e} = \frac{e_k - e_{k-1}}{\Delta t} \quad (2)$$

donde $\text{sgn} = +1$ para seguimiento de pared derecha y -1 para pared izquierda, de forma que un error positivo (el robot está demasiado cerca) siempre lo haga girar alejándose de la pared que sigue. El término $\delta = \text{deadband} = 0,03\text{m}$ evita que el robot “surfee” alrededor del punto de ajuste, y $k_p = 0,70$, $k_d = 1,60$ son las ganancias por defecto. La velocidad lineal se reduce de forma gradual entre $v_{\text{min}} = 0,06$ y $v_{\text{max}} = 0,18\text{m/s}$ cuando el frente cae bajo `front_slow = 0,50m`. La distancia objetivo a la pared (`wall_target`) y los umbrales de bloqueo frontal o lateral no son constantes fijas: se calculan a partir de la geometría física del robot (`robot_width = 0,16\text{m}`, `robot_length = 0,22\text{m}`) y de la separación deseada al borde (`wall_clearance = 0,12\text{m}`), de modo que el mismo nodo funcione sin recalibrarse sin importar el ancho real del pasillo el día de la prueba.

Sobre ese controlador base agregamos una corrección opcional por regresión de línea (`fit_wall_line`, activa por defecto con `line_fit_enabled=True`): en vez de leer

la distancia lateral de un único sector, ajustamos por mínimos cuadrados una recta a todos los puntos del `/scan` dentro de una ventana angular de 70° a 110° respecto al lado seguido (`line_window_lo_deg/line_window_hi_deg`), con rechazo iterativo de valores atípicos (hasta `line_outlier_iter= 3` iteraciones, descartando puntos con residuo mayor a `line_outlier_residual_m= 0,03m`) para no dejar que los puntos de una esquina cercana sesguen el ajuste. Cuando hay al menos `line_min_points= 6` puntos válidos dentro de un rango corto (`line_max_range_m= 0,55m`), la corrección angular se calcula sumando el error de ángulo de la recta ajustada y el error de distancia perpendicular ($k_{\text{ángulo}} = 1,2$, $k_{\text{distancia}} = 0,70$), lo que resulta más robusto al ruido que un solo rayo; si no hay suficientes puntos (pasillo abierto, esquina), el nodo cae de vuelta automáticamente al controlador PD de sector descrito arriba. También agregamos dos factores de calibración de escala del odómetro, `factor_dist_odom` y `factor_ang_odom` (ambos en 1,0 por defecto), para corregir un posible sesgo sistemático de `/odom_raw` una vez medido en pista, sin tener que recalibrar el resto de los umbrales.

III-D. Detección de intersecciones y callejones sin salida

Los sectores de decisión se calculan con `sector_robust`, que, a diferencia de tomar directamente el mínimo del sector, descarta primero los `sector_drop= 2` rayos más cercanos antes de calcular el mínimo. Esto evita que un solo rayo con una lectura espuria, ruido típico del MS200, dispare una parada o un giro falso. El sector frontal cubre $\pm 12^\circ$ y los sectores laterales van de 50° a 100° a cada lado. Una apertura (intersección o esquina) se detecta cuando la distancia a la pared seguida crece por encima de `wall_lost= 0,55m` (acotada de forma adaptativa a `corridor_width-0,05`). Un callejón sin salida se detecta cuando el frente y ambas paredes laterales caen a la vez bajo `wall_block`, y además exigimos que esa condición se mantenga durante `recover_persist= 1,0s` antes de disparar el giro de 180° , para no reaccionar a un roce momentáneo.

III-E. Detección de PARE por cámara (`pare_detector`)

El algoritmo es clásico y barato en cómputo, así que corre bien en el Raspberry Pi 5. Se apoya en OpenCV [4], una librería estándar de visión por computador, y procesa cada cuadro de `/image_raw` en cuatro pasos:

1. Convertimos la imagen de BGR a HSV, un modelo de color donde el matiz, la saturación y el brillo están separados, lo que facilita segmentar por color sin que la iluminación arruine el resultado, y segmentamos el rojo con dos bandas, porque en OpenCV el tono rojo envuelve los extremos 0 y 179 del canal H: $H \in [0, 10]$ y $H \in [170, 179]$, ambas con $S \geq 90$, $V \geq 60$.
2. Aplicamos una limpieza morfológica (una apertura seguida de un cierre, con un núcleo 5×5 , para quitar ruido y rellenar huecos pequeños) y extraemos los contornos



Figura 4. Prueba de calibración HSV: el algoritmo aísla la forma roja del cartel PARE sostenido frente a la cámara y superpone el contorno detectado (`/pare/debug_image`).

externos con el algoritmo de seguimiento de bordes de Suzuki y Abe (`cv2.findContours`) [5].

3. Filtramos el contorno candidato por su geometría: área mayor o igual a 350 px^2 , relación de aspecto ancho/alto entre 0,55 y 1,8 (para descartar franjas delgadas), *extent* (el área dividida entre la caja delimitadora) de al menos 0,45, y *solidity* (el área dividida entre la del casco convexo) de al menos 0,75. Este último filtro es el que descarta siluetas no convexas, como ropa o piel, que tienen entrantes y por eso bajan su *solidity*, mientras que un octágono o rombo real es casi convexo.
4. Aplicamos un *debounce* temporal: solo publicamos `True` en `/pare_detectado` si al menos el 60 % (`debounce_ratio= 0,6`) de los últimos 5 cuadros (`debounce_frames= 5`) tuvo una detección válida. Así evitamos que un falso positivo de un solo cuadro, por ejemplo un reflejo, dispare una parada.

También implementamos la “zona de atención” que pide el reto: si `use_attention_gate=True`, la detección solo se evalúa mientras `maze_solver` reporta, a través del tópico `/cerca_interseccion`, que el robot se está acercando a una intersección. Esa condición se calcula como frente por debajo de `front_slow` o pared seguida por encima del 85 % de `wall_lost`, y sirve para reducir falsos positivos en tramos rectos largos donde no puede haber un PARE. El nodo también publica `/pare/debug_image`, una imagen con la forma roja real resaltada (no un rectángulo genérico) y la etiqueta “PARE”, que se puede ver con `rqt_image_view` (Fig. 4 y Fig. 8).

III-F. Censo y rodeo de karpinchus (`box_detector`)

Para la variante opcional con obstáculos, `box_detector` segmenta el `/scan` en tramos continuos (un salto de rango mayor a 0,12m separa dos tramos), calcula el ancho físico de cada tramo multiplicando la distancia por el arco angular que cubre, y lo clasifica como caja si ese ancho cae en $0,20 \pm 0,08 \text{ m}$ y además sobresale al menos

MIN_PROTRUSION= 0,05 m respecto al fondo de la pared vecina. Cada caja detectada se proyecta a coordenadas del mundo usando la pose de /odom_raw, y se descarta como duplicado si cae a menos de BOX_MERGE_DIST= 0,30 m de una caja ya censada. El conteo se publica en /cajas_avistadas (que maze_solver usa para el contador karpinchus_rodados de la r brica) y las posiciones en /cajas_avistadas_pos (PoseArray, para monitoreo). Dentro de maze_solver, un obst culo frontal localizado (su centro est  m s cerca que los hombros laterales del robot, localized_front_obstacle) dispara una sub-m quina de estados de esquiv  (OUT→PASS→BACK, o retoma el rumbo directamente) que se detiene box_stop_wait_t= 3,0s frente a la caja antes de rodearla, sin perder el rumbo previo ni saltarse un PARE pendiente.

III-G. Fusión y arbitraje c mara-LiDAR

La regla de arbitraje del reto la implementamos de forma literal y con prioridad absoluta, salvo por la emergencia de colisi n. Si /pare_detectado es verdadero y no estamos en el periodo de enfriamiento (pare_cooldown_t= 2,5s desde el  ltimo PARE respetado, para no volver a dispararse sobre el mismo cartel), la FSM entra en PARAR_PARE, publica velocidad cero y se queda as  exactamente pare_wait_t= 3,0s, y al terminar incrementa el contador pare_respetados. Todo esto ocurre sin importar lo que indiquen los sectores del LiDAR. Lo  nico que puede anteponerse es la guarda de emergencia por colisi n inminente (emerg_dist= 0,15 m), de forma que un PARE nunca termine provocando un choque. Esto materializa exactamente la regla que pide el reto: la c mara manda para detener, el LiDAR manda para moverse y centrar.

III-H. Capas de seguridad

El nodo implementa varias guardas independientes de la l gica de navegaci n: se detiene por completo si el /scan tiene m s de scan_timeout= 0,5s de antig edad, porque eso significar a estar manejando a ciegas, o si la odometr a tiene m s de odom_timeout= 1,0s, porque su pose ya estar a desactualizada. Tambi n hay un watchdog de estancamiento posicional: si el robot pasa stuck_t= 2,5s sin avanzar al menos stuck_dpos= 0,03 m, dispara una reversa de recovery_t= 1,0s. Por  ltimo, antes de intentar rotar en el sitio, el nodo comprueba que el pasillo sensado en vivo sea f sicamente suficiente, es decir, al menos tan ancho como la diagonal del robot (aproximadamente 0,27 m), para no terminar en el estado FAULT por un timeout de giro en un pasillo demasiado angosto.

III-I. M tricas y monitoreo

Al llegar a META (dentro de un radio de meta_radius= 0,35 m alrededor del punto meta_x, meta_y en el marco de /odom_raw) o al cerrar el nodo, maze_solver escribe una fila en metrics_granprix.csv con exactamente las columnas

Cuadro I
COLUMNAS DE METRICAS_GRANPRIX.CSV (IMPLEMENTADAS EN
MAZE_SOLVER.WRITE_METRICS)

Campo	Origen
ronda, llego_meta	par�metro / evento META
tiempo_s	reloj interno desde 1�r /odom
long_ruta_cm	integraci�n de /odom_raw
long_optima_cm	medici�n manual sobre el plano
eficiencia	long_optima_cm / long_ruta_cm
colisiones, pare_falsos	manual (video / ros2 bag)
pare_reales	par�metro (conteo real en pista)
pare_detectados, pare_respetados	contadores de on_pare/FSM
dead_ends_visitados	contador de eventos RECOVER
karpinchus_rodados	contador de sub-FSM de esquiv�

que pide la r brica del reto: ronda, llego_meta, tiempo_s, long_ruta_cm (integrada a partir de la odometr a, descartando saltos > 0,5 m como ruido), long_optima_cm (medida a mano sobre el plano), eficiencia (el cociente entre long_optima_cm y long_ruta_cm), colisiones, pare_reales, pare_detectados, pare_respetados, pare_falsos, dead_ends_visitados y karpinchus_rodados (Tabla I). Los campos que no se pueden medir de forma autom tica, como colisiones y los falsos PARE, quedan marcados como pendientes de completar por revisi n de video, tal como pide el propio reto. En paralelo, el nodo publica /maze_state (un texto legible con el estado en vivo), marcadores de RViz en /granprix_markers para cada intersecci n, PARE y META, y atiende /dashboard_pause, que permite pausar el robot manualmente desde el panel web sin que pierda el estado de la FSM.

IV. CONFIGURACI N EXPERIMENTAL

IV-A. Plataforma rob tica y sensores

El robot es una base Yahboom Micro-ROS sobre Raspberry Pi 5, con un LiDAR 2D MS200 (~10 Hz) montado en una torreta y una c mara USB sobre un soporte pan orientable en el frente (Fig. 3). El robot expone /scan (sensor_msgs/LaserScan), /odom_raw (nav_msgs/Odometry, con una configuraci n de calidad de servicio BEST_EFFORT/VOLATILE t pica de los drivers embebidos, que en la pr ctica prioriza mensajes recientes sobre no perder ninguno) e /image_raw (sensor_msgs/Image), y recibe comandos de velocidad en /cmd_vel (geometry_msgs/Twist).

IV-B. Pista: el laberinto f sico

La pista se construy  con tablas de MDF de 30 × 60 cm puestas de canto sobre una rejilla de 6 × 4 celdas de 60 cm (360 × 240 cm en total), con la apertura de INICIO en el borde izquierdo y META en la esquina opuesta, varias rutas v lidas y callejones sin salida puestos a prop sito (Fig. 5). Las se ales PARE son carteles octagonales rojos sobre un soporte

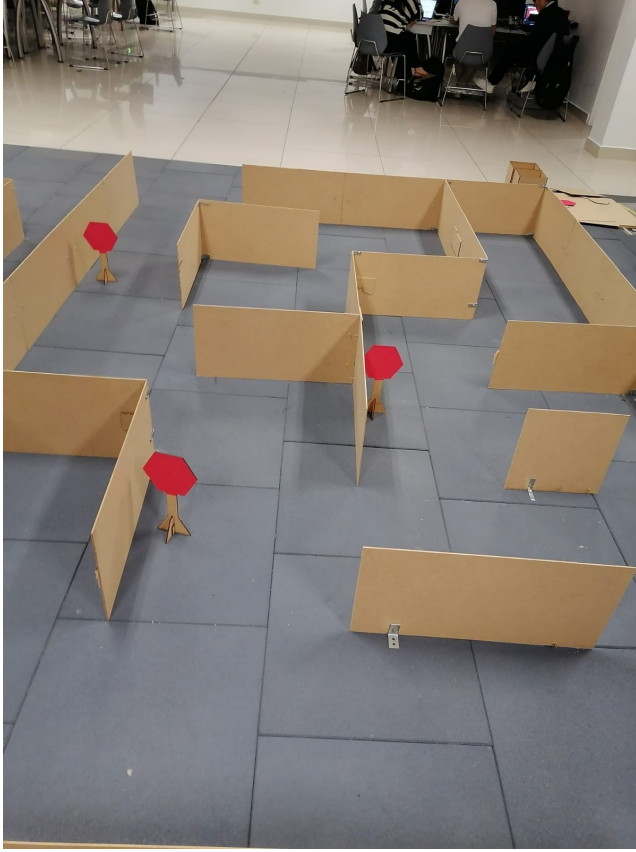


Figura 5. Vista aérea de la pista construida: paredes de MDF de canto sobre rejilla 6×4 de 60 cm, con dos señales PARE visibles en distintas intersecciones.

Cuadro II
PARÁMETROS DE EJECUCIÓN (CONFIG/GRANPRIX_PARAMS.YAML)

Nodo	Parámetro	Valor
maze_solver	side (regla de mano)	right
maze_solver	pare_wait_t / cooldown	3.0 s / 2.5 s
maze_solver	meta_radius	0.35 m
maze_solver	enable_obstacle_veer	true
pare_detector	min_area	350 px ²
pare_detector	aspect [lo, hi]	[0.55, 1.8]
pare_detector	min_extent	0.45
pare_detector	debounce [frames, ratio]	[5, 0.6]
box_detector	BOX_SIZE $\pm$ TOL	0.20 $\pm$ 0.08 m

de madera, colocados a la altura del campo de visión de la cámara en algunas intersecciones (Fig. 6).

IV-C. Parámetros de ejecución

La Tabla II resume los parámetros de más alto nivel, que se pueden editar entre corridas desde config/granprix_params.yaml sin tocar el código. Los umbrales de más bajo nivel del controlador (el PD, las aperturas, el callejón sin salida) ya vienen afinados en los DEFAULTS de maze_solver.py, y se calculan de forma adaptativa a partir de la geometría del robot (Sección III-C).

IV-D. Protocolo de pruebas

Antes de las corridas de competencia calibramos los rangos HSV directamente en la cancha, con el modo -calibrar de pare_detector.py (sliders en vivo sobre el cartel real, bajo la iluminación del salón), y validamos cada nodo por separado: pare_detector sosteniendo el cartel frente a la cámara (Fig. 4), maze_solver recorriendo tramos del laberinto en modo de seguimiento de pared, y scan_map_viewer/web_dashboard para verificar en vivo cómo se clasifica el /scan entre PARED y CAJA, además de la trayectoria acumulada por odometría (Fig. 9). El protocolo de competencia contempla dos rondas sobre la misma pista, reconfigurada entre una y otra: la Ronda 1 es de exploración, sin conocer el trazado, y la Ronda 2 es de *time attack*, buscando la ruta más corta con cero colisiones y todos los PARE respetados.

V. RESULTADOS

V-A. Validación funcional por componente

Las pruebas experimentales realizadas sobre la pista física permitieron verificar el funcionamiento de los principales módulos del sistema. La detección de señales PARE se mantuvo estable tanto durante la etapa de calibración (Fig. 4) como en las pruebas realizadas dentro del laberinto (Fig. 8), donde el algoritmo identificó correctamente el contorno del cartel mediante la segmentación en el espacio HSV y los filtros geométricos implementados. El uso del criterio de solidity contribuyó a reducir falsas detecciones ocasionadas por objetos de color similar presentes en el entorno.

Asimismo, las pruebas de navegación evidenciaron que el controlador de seguimiento de pared mantuvo una trayectoria estable en los pasillos del laberinto. Como se observa en la Fig. 6, el robot logró desplazarse manteniendo una distancia lateral consistente respecto a la pared mientras la cámara supervisaba la detección de señales PARE en las intersecciones; el panel de monitoreo web (Fig. 7) permitió verificar en vivo esa misma trayectoria y la segmentación del /scan. Estos resultados confirman el correcto funcionamiento de la estrategia de integración entre la información del LiDAR y la cámara durante la navegación.

V-B. Estado de las métricas de competencia

El esquema de metricas_granprix.csv (Tabla I) está implementado y verificado: escribe una fila completa tanto al llegar a META como al cerrar el nodo manualmente, así que ninguna corrida queda sin registrar. Sin embargo, al momento de escribir este artículo los parámetros meta_enabled, long_optima_cm y pare_reales de config/granprix_params.yaml siguen en sus valores por defecto (false, 0.0 y 0, respectivamente), pendientes de medirse sobre la configuración real de la pista el día de cada corrida oficial, tal como indica el propio comentario del archivo de configuración. Por eso este artículo reporta evidencia de **validación funcional** de cada componente (detección de PARE, seguimiento de pared, censo de cajas, monitoreo en vivo), pero todavía no las métricas cuantitativas de las

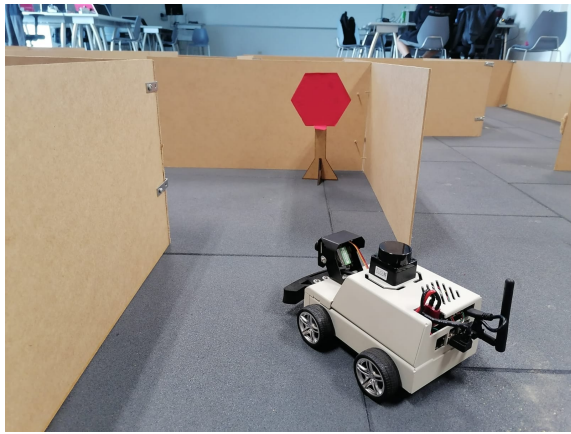


Figura 6. Robot aproximándose a una señal PARE dentro de un pasillo del laberinto durante el seguimiento de pared por LiDAR.

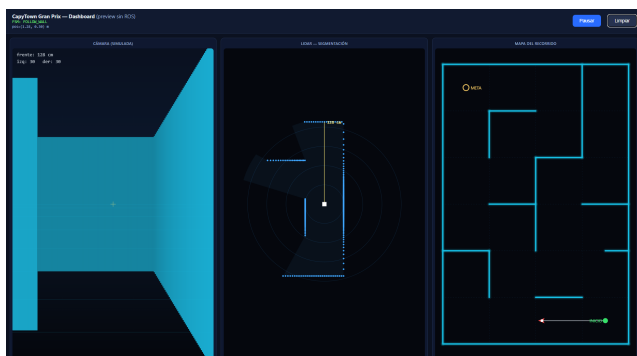


Figura 7. Panel de monitoreo web (web_dashboard): cámara, /scan segmentado y mapa del recorrido (INICIO/META) en una sola pestaña de navegador, con botón de pausa.

Rondas 1 y 2 (tiempo_s, eficiencia, colisiones), que completaremos en las corridas de competencia con la pista y las señales PARE ya reconfiguradas.

VI. DISCUSIÓN

El sistema que implementamos es deliberadamente reactivo: sigue paredes y decide en cada intersección con la información local del /scan, en vez de construir primero un mapa de ocupación [6] del laberinto completo y planificar sobre él con un algoritmo de búsqueda de camino mínimo como A* [7]. Un mapa de ocupación es, básicamente, una cuadrícula donde cada celda guarda qué tan probable es que esté ocupada por una pared, y A* es un algoritmo que busca el camino más corto sobre ese mapa. Elegimos no usarlos porque evitan la complejidad y el costo de cómputo de mantener un mapa consistente bajo el ruido de la odometría por *dead-reckoning* [8], es decir, estimar la posición solo a partir del movimiento de las ruedas, sin ninguna referencia externa que corrija el error acumulado, algo especialmente costoso en el Raspberry Pi 5. A cambio, no garantizamos en general la ruta más corta en la Ronda 2, salvo que la topología del laberinto nos favorezca. El diseño prioriza la robustez ante las condiciones reales de la pista física por encima de la optimalidad teórica.

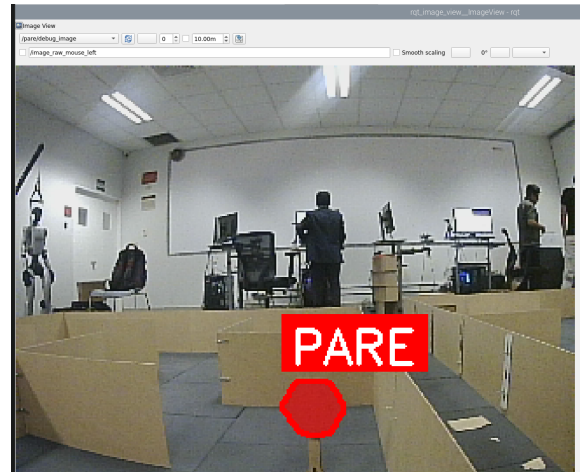


Figura 8. Detección de PARE en vivo dentro del laberinto (rqt_image_view sobre /pare/debug_image): el contorno real del cartel es resaltado y etiquetado.

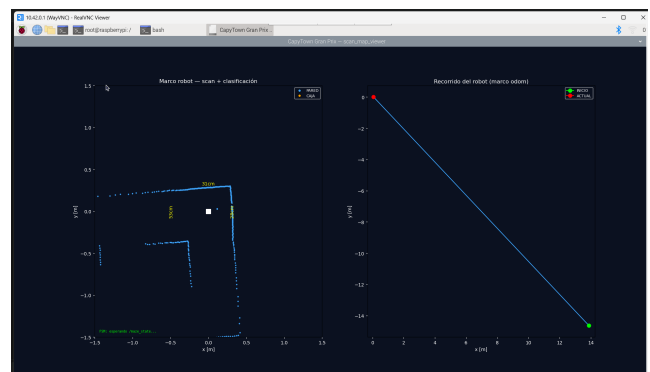


Figura 9. Panel de monitoreo (scan_map_viewer): a la izquierda, el /scan clasificado en PARED/CAJA en el marco del robot con las distancias de sector anotadas; a la derecha, la trayectoria acumulada por odometría desde INICIO.

Tres decisiones ilustran ese compromiso. La primera es anclar los umbrales de seguimiento de pared a la geometría del robot (W , L) en vez de a las dimensiones nominales del laberinto: así, el mismo nodo funciona sin recalibrarse aunque el ancho real del pasillo termine siendo distinto al del plano. La segunda es el filtro de *solidity* en la detección de PARE, en vez de quedarnos solo con área y relación de aspecto. Lo agregamos específicamente porque siluetas convexas irregulares, como ropa o manos, pueden pasar un filtro de área y aspecto sin problema, pero casi nunca imitan la convexidad de un octágono real, así que este filtro reduce los falsos positivos sin necesidad de entrenar ningún clasificador. La tercera es que implementamos la regla “la cámara manda para detener, el LiDAR manda para moverse” como una interrupción de máxima prioridad en el bucle de control, antes que cualquier lógica de seguimiento de pared, y no como un estado más de la FSM geométrica. Lo hicimos así justamente para que la lógica de navegación nunca pueda “competir” con una parada de seguridad que la cámara ya confirmó.

La principal limitación por ahora es que el sistema todavía no ha corrido las dos rondas oficiales sobre la pista con la geometría y las señales PARE definitivas. Los umbrales de detección de intersección y de censo de cajas ya están afinados con datos de campo, pero la eficiencia de ruta y la tasa de PARE respetados sin falsos positivos, que son los dos criterios de mayor peso en la rúbrica de implementación, solo se pueden reportar con datos reales de esas corridas. Otra limitación que reconocemos en el propio código es que `colisiones` y `pare_falsos` no son cosas que el robot pueda detectar por sí solo, así que dependen de revisar manualmente el video o el `ros2 bag` de cada corrida.

VII. CONCLUSIÓN

En este trabajo se desarrolló un sistema de navegación autónoma para el reto CapyTown Gran Prix, integrando sensores LiDAR y cámara RGB sobre una plataforma Yahboom Micro-ROS con ROS 2. La arquitectura implementada permitió dividir las funciones principales en módulos independientes para la navegación, la detección de señales PARE y la evasión de obstáculos, facilitando tanto el desarrollo como las pruebas de cada componente.

Las pruebas realizadas muestran que el sistema responde de manera estable durante el seguimiento de paredes y es capaz de detectar correctamente las señales PARE bajo las condiciones evaluadas. Asimismo, la integración de la información proveniente de la cámara y el LiDAR permitió mejorar la toma de decisiones durante la navegación dentro del laberinto.

Si bien el sistema cumple con los objetivos planteados para esta etapa del proyecto, aún queda pendiente validar su desempeño durante las corridas oficiales de competencia y analizar las métricas finales de tiempo, eficiencia y precisión. Como trabajo futuro, se plantea optimizar los parámetros de control y continuar realizando pruebas en diferentes condiciones para aumentar la robustez y confiabilidad del sistema.

RESUMEN PARA LA SUSTENTACIÓN (EN LENGUAJE SIMPLE)

El robot tiene que cruzar solo un laberinto de tableros, desde un punto de inicio hasta una meta, sin que nadie lo maneje. Para “ver” usa dos sensores que se complementan. El LiDAR es como un radar láser que gira y mide qué tan lejos está cada pared a su alrededor; con esos datos, el robot sabe si tiene pared a la derecha, si el frente está libre, si llegó a una esquina o si se metió en un callejón sin salida. La cámara, en cambio, no mide distancias: reconoce colores y formas, y su único trabajo es detectar el cartel rojo de PARE. Usamos dos sensores distintos porque cada uno resuelve un problema que el otro no puede: el LiDAR nunca podría “leer” un cartel, y la cámara sola no sabría seguir una pared con precisión. Por dentro, el robot funciona como una máquina de estados: en cada instante está en un solo modo (seguir pared, girar, recuperarse de un callejón sin salida, o parado en un PARE) y va saltando de un modo a otro según lo que detecta el LiDAR. Para seguir la pared usamos un control PD, que es simplemente una fórmula que corrige la trayectoria según qué tan lejos está de la distancia ideal a la pared, y

también según qué tan rápido se está alejando o acercando, para que el giro sea suave y no oscile de un lado a otro. La regla más importante del sistema es el arbitraje entre los dos sensores: la cámara manda cuando hay que parar (si detecta un PARE, el robot se detiene tres segundos sí o sí, sin importar lo que diga el LiDAR), pero el LiDAR manda cuando hay que moverse y mantenerse centrado en el pasillo. Esa prioridad está programada como una interrupción que se revisa antes que cualquier otra decisión, para que las dos señales nunca puedan “pelearse”. Además, el sistema tiene varias capas de seguridad por si algo falla: se detiene si los datos del sensor están muy viejos, si deja de avanzar por un rato, o si está a punto de chocar. Todo lo que el robot hace se guarda en un archivo de métricas, para poder evaluar después qué tan eficiente fue la ruta y cuántos PARE respetó. En resumen: el LiDAR le da al robot los “ojos” para moverse por el laberinto, la cámara le da el “criterio” para respetar las reglas del camino, y la máquina de estados es la que decide, en cada momento, a cuál de los dos hacerle caso.

REFERENCIAS

- [1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot Operating System 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, 2022.
- [2] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2nd ed. MIT Press, 2011.
- [3] T. Bräunl, *Embedded Robotics: Mobile Robot Design and Applications with Embedded Systems*, 3rd ed. Springer, 2008.
- [4] G. Bradski, “The OpenCV library,” *Dr. Dobbs’s Journal of Software Tools*, 2000.
- [5] S. Suzuki and K. Abe, “Topological structural analysis of digitized binary images by border following,” *Computer Vision, Graphics, and Image Processing*, vol. 30, no. 1, pp. 32–46, 1985.
- [6] A. Elfes, “Using occupancy grids for mobile robot perception and navigation,” *Computer*, vol. 22, no. 6, pp. 46–57, 1989.
- [7] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [8] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005.